School of Future Tech

Case Study Report

on

Inventory Management System

by

Saiyash Poojari

150096725136

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Inventory management plays a crucial role in businesses such as retail stores, warehouses, and supply chains. Efficient management of products ensures that stock levels are properly maintained, preventing shortages or overstocking. Managing inventory manually can be time-consuming and error-prone, especially when dealing with multiple products and frequent stock updates.

This case study focuses on designing and implementing an **Inventory Management System using C++**. The system allows users to add product details, update stock quantities, and display the current inventory. It is implemented as a **menu-driven console application**, enabling users to interact with the system dynamically during execution.

The project demonstrates how programming concepts such as **structures, arrays, loops, and conditional statements** can be used to develop a simple yet practical business application. It provides students with hands-on experience in managing structured data and implementing logical program flow.

2. Problem Statement / Case Background (Abstract)

Background

In many small businesses and shops, inventory is often managed manually or using basic record systems. This can lead to problems such as incorrect stock counts, difficulty tracking product information, and delays in updating stock levels.

A simple computerized inventory system can significantly improve efficiency by maintaining accurate records and allowing quick updates. By automating inventory operations, businesses can reduce human errors and make better decisions regarding stock management.

Abstract

This case study presents the design and implementation of a **menu-driven Inventory Management System using C++**. The system stores product information including **Product ID, Product Name, Quantity, and Price**.

Users can perform operations such as adding new products, updating stock quantities, and viewing the inventory report. The program uses **structures to organize product data, arrays to store multiple products, loops for repeated operations, and conditional statements for decision making**.

Additionally, the system calculates the **total value of inventory**, which helps in understanding the overall worth of products stored in the inventory.

3. Case Study Design

The Inventory Management System is designed as a **console-based menu-driven application** with the following functional stages:

1. Product Data Storage

The system stores product information using a **structure (struct)** that includes:

- Product ID
- Product Name
- Quantity
- Price

Multiple products are stored using an **array of structures**.

2. Add Product

This feature allows the user to enter product details and store them in the inventory system.

Steps:

- User enters Product ID
- User enters Product Name
- User enters Quantity
- User enters Price
- Product is stored in the inventory array

3. Update Stock

This function allows updating the stock of an existing product.

Steps:

- User enters Product ID
- System searches for the product
- User enters additional quantity
- Quantity is updated

4. View Inventory Report

This feature displays all stored product details including:

- Product ID
- Product Name
- Quantity
- Price
- Total value of each product

It also calculates the **total value of the entire inventory**.

5. Exit System

The program terminates when the user selects the exit option.

4. Methods & Algorithms Technology Applied in the Case Study

Key Programming Methods Used

1. Data Structure (Structure)

A **structure (struct)** is used to group related data such as product ID, name, quantity, and price into a single data type.

Example:

```
struct Product

{

    int id;
```

```
    string name;

    int quantity;

    float price;

};
```

2. Arrays

An **array of structures** is used to store multiple products.

Example:

```
Product inventory[100];
```

This allows storing up to **100 products**.

3. Loops

Loops are used to:

- Display all products
- Search products
- Repeat the menu until the user exits

Example: Do while loop is used in this program.

4. Conditional Statements

Conditional statements are used to check conditions such as:

- Product ID match
- Menu choice selection
- Valid input

Example:

```
if(inventory[i].id == searchID)
```

5. Menu-Driven Programming

The program uses a **do-while loop** to repeatedly display the menu until the user selects the exit option.

Technology Stack Used

- Programming Language: **C++**

- Development Environment: **VS Code**
- Compiler: **GCC / G++**
- Platform: **Console Application**

5. Problem Statement / Case Study Implementation Details and Snapshots

The implementation of the Inventory Management System consists of a single C++ program that performs all required operations.

Main Functional Modules

1. **Add Product Module**
    - **Accepts product details from the user.**
    - **Stores them in the inventory array.**
2. **Update Stock Module**
    - **Searches product using Product ID.**
    - **Updates the quantity of the selected product.**
3. **Display Inventory Module**
    - **Displays all stored products.**
    - **Calculates total value of each product.**
4. **Menu Control Module**
    - **Allows the user to select operations repeatedly.**

Snapshots:

Code Snippets:

inventory.cpp > main()

```cpp
#include <iostream>
using namespace std;

struct Product {
    int id;
    string name;
    int quantity;
    float price;
};

int main() {

    Product inventory[100];
    int count = 0;
    int choice;

    do {

        cout << "\n===== Inventory Management System =====\n                    By Saiyash Poojari\n";
        cout << "1. Add Product\n";
        cout << "2. Update Stock\n";
        cout << "3. View Inventory\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch(choice) {

        case 1:

            cout << "\nEnter Product ID: ";
            cin >> inventory[count].id;

            cout << "Enter Product Name: ";
            cin >> inventory[count].name;

            cout << "Enter Quantity: ";
            cin >> inventory[count].quantity;

            cout << "Enter Price of Single Item: ";
            cin >> inventory[count].price;

            count++;

            cout << "Product Added Successfully!\n";
```

```
            break;

        case 2:
        {
            int searchID;
            cout << "\nEnter Product ID to update stock: ";
            cin >> searchID;

            bool found = false;

            for(int i = 0; i < count; i++) {

                if(inventory[i].id == searchID) {

                    int addQty;

                    cout << "Enter quantity to add: ";
                    cin >> addQty;

                    inventory[i].quantity += addQty;

                    cout << "Stock Updated Successfully!\n";
                    cout << "Current Quantity: " << inventory[i].quantity << endl;

                    found = true;
                    break;
                }
            }

            if(!found) {
                cout << "Product not found!\n";
            }

            break;
        }

        case 3:
        {
            cout << "\n===== Inventory Report =====\n";

            float totalInventoryValue = 0;

            if(count == 0) {
                cout << "No products available.\n";
            }
```

```
            else {

                for(int i = 0; i < count; i++) {

                    float productValue = inventory[i].quantity * inventory[i].price;

                    cout << "\nProduct ID: " << inventory[i].id;
                    cout << "\nName: " << inventory[i].name;
                    cout << "\nQuantity: " << inventory[i].quantity;
                    cout << "\nPrice: " << inventory[i].price;
                    cout << "\nTotal Value: " << productValue;
                    cout << "\n----------------------";

                    totalInventoryValue += productValue;
                }

                cout << "\n\nTotal Inventory Value: " << totalInventoryValue << endl;
            }

            break;
        }

        case 4:
        cout << "Exiting Program...\nPROGRAM TERMINATED SUCCESSFULLY!\n";
            break;

        default:
            cout << "Invalid choice!\n";
        }

    } while(choice != 4);

    return 0;
}
```

Output:

```
saiyashpoojari@Saiyash-MacBook-Air Saiyash C++ % cd "/Users/saiyashpoojari/Desktop/Saiyash C++/" && g++ inventory.cpp -o inventor
y && "/Users/saiyashpoojari/Desktop/Saiyash C++/"inventory

===== Inventory Management System =====
                    By Saiyash Poojari
1. Add Product
2. Update Stock
3. View Inventory
4. Exit
Enter your choice:
```

```
===== Inventory Management System =====
                    By Saiyash Poojari
1. Add Product
2. Update Stock
3. View Inventory
4. Exit
Enter your choice: 1

Enter Product ID: 100
Enter Product Name: Mobile
Enter Quantity: 2
Enter Price of Single Item: 20000
Product Added Successfully!
```

```
===== Inventory Management System =====
                    By Saiyash Poojari
1. Add Product
2. Update Stock
3. View Inventory
4. Exit
Enter your choice: 2

Enter Product ID to update stock: 100
Enter quantity to add: 3
Stock Updated Successfully!
Current Quantity: 5
```

```
Enter your choice: 3

===== Inventory Report =====

Product ID: 100
Name: Mobile
Quantity: 5
Price: 20000
Total Value: 100000
----------------------
```

```
===== Inventory Management System =====
                    By Saiyash Poojari
1. Add Product
2. Update Stock
3. View Inventory
4. Exit
Enter your choice: 4
Exiting Program...
PROGRAM TERMINATED SUCCESSFULLY!
```

6. Problem Statement / Case Study Results and Conclusion

Findings

The developed Inventory Management System successfully performs the following operations:

- Adding product details
- Updating stock quantities
- Displaying inventory records
- Calculating total inventory value

The use of structures and arrays allows efficient storage and retrieval of product information.

Conclusion

This case study demonstrates the implementation of a simple yet effective **Inventory Management System using C++**. The system shows how fundamental programming concepts such as **structures, arrays, loops, and conditional statements** can be used to develop practical applications.

The project provides a basic framework that can be further expanded to include advanced features such as file storage, graphical interfaces, and database integration. Overall, the system helps in understanding how programming can be applied to solve real-world inventory management problems.

7. References

- C++ Programming Documentation
- Standard C++ Library Reference
- Online tutorials on C++ structures and arrays
- Programming textbooks related to C++ and data structures